

Clavra ProdPilot™

Software Requirements Specification (SRS)

Version 1.1 | June 2026 | Includes AI Vision Module

Document Title	Clavra ProdPilot™ – SRS
Version	1.1 (added: Section 3.9 AI Vision Analysis)
Date	June 12, 2026
Status	Active Development
Stack	FastAPI + React 18 + PostgreSQL + pgvector + GPT-4o + GPT-4o Vision + Ollama
Target Audience	Developers, QA Engineers, Stakeholders

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for Clavra ProdPilot™, an AI-powered ERP for garment manufacturing. It is the authoritative reference for development, testing, and stakeholder alignment.

1.2 Scope

The system covers: production order management, shipment tracking, raw material inventory, production line monitoring, quality control, AI Copilot (text + voice), RAG document retrieval, and AI Vision for automated quality inspection via image analysis.

1.3 Definitions

Term	Definition
ERP	Enterprise Resource Planning – integrated management of core business processes
RAG	Retrieval-Augmented Generation – LLM answers grounded in retrieved documents
Intent	Classified purpose of a user message (e.g. shipment_status, defect_analysis)
pgvector	PostgreSQL extension for storing and querying 1536-dim vector embeddings
SOP	Standard Operating Procedure – step-by-step work instruction document
RBAC	Role-Based Access Control – permissions tied to Admin/Manager/Operator roles
JWT	JSON Web Token – stateless authentication token
Ollama	Local LLM inference server (llama3.1:8b text, moondream/llava vision)
moondream	Lightweight local vision model (~1.6 GB) for offline image analysis
llava	LLaVA – secondary local vision model (~4 GB) used as fallback if moondream absent
defect_analysis	AI intent that routes to the Vision service for image-based QC inspection

2. Overall Description

2.1 Product Perspective

Clavra ProdPilot™ is a standalone web application replacing spreadsheet-based factory workflows. It integrates with OpenAI for cloud AI (text + vision) and Ollama for local AI (text via llama3.1:8b, vision via moondream/llava), enabling fully offline deployments.

2.2 User Classes

User Class	Role	Primary Tasks
Factory Admin	Full access	Configure org, manage users, view all reports
Production Manager	Manager	Monitor orders, update progress, view analytics
QC Inspector	Operator	Upload defect images, log QC results, use AI Vision
Line Supervisor	Operator	Update line status, record quantities
Logistics Officer	Operator	Manage shipments, link orders

2.3 Operating Environment

- Server: Linux/Windows with Python 3.11+, PostgreSQL 16+, Ollama
- Client: Chrome 120+ / Firefox 120+ / Edge 120+ with camera/microphone access
- Vision (cloud): OpenAI GPT-4o Vision – requires API key with vision credits
- Vision (local): Ollama moondream (~1.6 GB) or llava (~4 GB) – no API key needed
- Docker: Full-stack via docker-compose + Nginx reverse proxy

3. Functional Requirements

3.1 Authentication & RBAC

ID	Requirement
FR-AUTH-01	Login with email+password returns JWT access token + refresh token
FR-AUTH-02	Refresh tokens rotate on each use; expired tokens rejected with HTTP 401
FR-AUTH-03	All endpoints (except /auth/login) require valid Bearer JWT
FR-AUTH-04	All data scoped to org_id – no cross-org data leakage
FR-AUTH-05	Roles: Admin > Manager > Operator – progressively fewer write permissions

3.2 Production Order Management

ID	Requirement
FR-PROD-01	CRUD for orders: order_no, buyer, style, colour, total_qty, status, delivery_date
FR-PROD-02	Status lifecycle: Draft → Cutting → Sewing → Finishing → QC → Packed → Shipped
FR-PROD-03	Record produced_qty and defect_qty; system calculates progress percentage
FR-PROD-04	Inline editing of progress, delivery date, and shipment link from UI
FR-PROD-05	Link order to shipment via PUT /orders/{id}/link-shipment

3.3 Production Line Management

ID	Requirement
FR-LINE-01	Define production lines (A/B/C/D) with capacity and current order assignment
FR-LINE-02	Real-time status: Idle / Running / Maintenance
FR-LINE-03	Track efficiency percentage per line

3.4 Shipment Management

ID	Requirement
FR-SHIP-01	CRUD for shipments: shipment_no, destination, carrier, status, estimated_delivery
FR-SHIP-02	Status lifecycle: Pending → In Transit → Delivered → Cancelled
FR-SHIP-03	Link/unlink production orders to shipments

3.5 Inventory Management

ID	Requirement
FR-INV-01	CRUD for raw materials: name, quantity, unit, reorder_level, supplier

ID	Requirement
FR-INV-02	Flag items where quantity <= reorder_level as low stock
FR-INV-03	Inventory analytics: trend charts, low-stock summary

3.6 AI Copilot

ID	Requirement
FR-AI-01	WebSocket /ai/ws streams responses token-by-token in real time
FR-AI-02	Intent pipeline: GPT-4o (primary) → keyword rules → Ollama (fallback)
FR-AI-03	18+ intents: general_status, get_order_status, shipment_status, production_line_status, ask_manual, ask_policy, inventory_check, defect_analysis, count_orders, and more
FR-AI-04	Pre-routing override guards correct LLM misclassifications before dispatch
FR-AI-05	Voice input via Web Speech API; TTS output via browser SpeechSynthesis
FR-AI-06	Intent badge and confidence score shown in UI for each response

3.7 RAG Knowledge Base

ID	Requirement
FR-RAG-01	Upload PDF, DOCX, TXT, XLSX documents up to 50 MB
FR-RAG-02	Documents chunked (512 words, 64 overlap), embedded with text-embedding-3-small
FR-RAG-03	Query: cosine similarity (pgvector, MIN_SIM=0.72); ILIKE fallback
FR-RAG-04	Answers: GPT-4o → Ollama fallback → formatted excerpt fallback
FR-RAG-05	Responses include source citations: document name + page number
FR-RAG-06	Re-index button regenerates all chunks and embeddings for a document

3.8 Analytics

ID	Requirement
FR-ANA-01	Dashboard KPIs: active orders, in-transit shipments, low stock, on-time %
FR-ANA-02	Order trend chart (last 30 days), production efficiency per line
FR-ANA-03	AI generates and executes SQL for count/sum/trend questions

3.9 AI Vision Analysis

The AI Vision module allows QC inspectors and shop-floor workers to photograph or upload images of garments, fabric, labels, or equipment directly inside the AI Copilot chat. Images are analyzed by a vision AI and a structured quality report is returned.

ID	Requirement
FR-VIS-01	Users upload images (JPEG, PNG, WEBP) via the camera button in the AI Copilot chat interface
FR-VIS-02	Image analysis triggered by defect_analysis intent – routed to vision_service.analyze_image()
FR-VIS-03	Analysis pipeline: GPT-4o Vision (cloud, high detail) → Ollama moondream (~1.6 GB local) → Ollama llava (~4 GB local) → graceful fallback message
FR-VIS-04	Supported analysis types: defect_detection label_reading line_photo equipment_check
FR-VIS-05	Every analysis returns structured JSON: analysis_type, findings[], severity (critical/major/minor/none), recommendations[], defect_rate_estimate, confidence, summary
FR-VIS-06	GPT-4o Vision uses json_object response format with VISION_PROMPT for structured output; Ollama response parsed via _parse_ollama_response()
FR-VIS-07	All analysis results persisted in vision_analyses table (org_id, user_id, image_filename, analysis_type, findings JSON, defect_rate, confidence, summary, created_at)
FR-VIS-08	System discovers available Ollama vision models dynamically via GET /api/tags – tries moondream first, llava second
FR-VIS-09	Graceful fallback: if no vision model available, returns user-friendly message with instructions to add OpenAI credits or pull moondream – never crashes
FR-VIS-10	Ollama vision calls wrapped with asyncio.wait_for(timeout=60s) to prevent blocking the event loop on slow local inference

Supported Analysis Types Detail:

Analysis Type	Trigger Keywords	What AI Looks For
defect_detection	defect, fault, damage, hole, stain, tear, seam, colour	Holes, stains, seam failures, colour inconsistencies, pilling, snags, weave defects
label_reading	label, tag, care label, size, country of origin	Care instructions, size markings, country-of-origin, brand labels, compliance marks
line_photo	production line, workstation, floor, line photo	Worker ergonomics, machine setup, material placement, workstation cleanliness
equipment_check	machine, equipment, sewing machine, needle, iron, cutter	Visible wear, needle condition, calibration issues, maintenance needs

4. Non-Functional Requirements

4.1 Performance

- REST API p95 response time < 500ms (non-AI endpoints)
- Intent classification < 2s (GPT-4o) / < 8s (Ollama text)
- WebSocket first-token latency < 1.5s
- Vision analysis: GPT-4o Vision < 10s; Ollama moondream < 60s (local hardware dependent)
- Vector similarity search < 200ms for 100K chunks

4.2 Security

- Passwords hashed with bcrypt (cost factor 12)
- JWT access tokens expire 30 min; refresh tokens 7 days with rotation
- All DB queries parameterised – SQL injection prevention
- Uploaded images validated by MIME type before processing
- Vision analysis results scoped to org_id – no cross-org leakage

4.3 Reliability

- AI Copilot: GPT-4o quota exceeded → keyword rules → Ollama text
- AI Vision: GPT-4o Vision → Ollama moondream → Ollama llava → graceful message
- RAG: vector search → ILIKE keyword fallback

4.4 Scalability

- Stateless FastAPI instances – horizontally scalable behind load balancer
- pgvector IVFFlat index for sub-linear nearest-neighbour search

5. System Architecture

5.1 High-Level Diagram

Browser (React/TS) → **HTTP/WS** → FastAPI + Uvicorn → **asyncpg** → PostgreSQL + pgvector

□ HTTPS → [OpenAI GPT-4o Text] [OpenAI GPT-4o Vision] [Ollama llama3.1:8b] [Ollama moondream / llava]

5.2 AI Text Intent Pipeline

- Stage 1 – detect_intent(): GPT-4o (8s timeout) → keyword rules → Ollama
- Stage 2 – route_intent(): 4 override guards → domain handler dispatch
- Stage 3 – Handler: DB query / RAG / SQL generation / Vision service

5.3 AI Vision Pipeline

- Step 1 – User uploads image via ImageUploadZone (camera button in chat)
- Step 2 – defect_analysis intent routes to vision_service.analyze_image()
- Step 3 – GPT-4o Vision tried first (json_object mode, high detail)
- Step 4 – If OpenAI unavailable: _find_ollama_vision_model() queries /api/tags
- Step 5 – moondream tried first, then llava, with 60s asyncio timeout
- Step 6 – _parse_ollama_response() normalises free-text into structured dict
- Step 7 – Result saved to vision_analyses table + returned as chat bubble

5.4 RAG Pipeline

- Ingest: upload → extract (PyMuPDF/docx/openpyxl) → chunk (512w/64 overlap) → embed → pgvector
- Query: embed question → cosine similarity (MIN_SIM=0.72, TOP_K=5) → keyword fallback → answer

6. Future Work & Recommendations

6.1 Short-Term (Next 3 Months)

Feature	Description	Priority
Bangla/Hindi UI	Multi-language support for local factory workers	High
Push Notifications	Alerts for order delays, low stock, delivery updates	High
Mobile PWA	Responsive mobile + offline mode with service workers	High
Export Reports	PDF/Excel export of production reports and shipment summaries	Medium
Vision Batch Mode	Upload multiple defect images at once for bulk QC inspection	Medium
Vision History Page	Dedicated page showing all past vision analyses with filters	Medium

6.2 Medium-Term (3–9 Months)

Feature	Description	Priority
Supplier Portal	External portal for suppliers to update material delivery status	High
Costing Module	Material + labour cost tracking, margin per production order	High
Barcode/QR Scan	Mobile camera for order tracking and inventory check-in	Medium
Vision Defect Stats	Dashboard chart: defect rates by order, line, garment type over time	Medium
Conveyor Camera	RTSP camera stream integration for automated line QC (no manual upload)	Medium
Email Alerts	Automated PO confirmations, dispatch alerts via SendGrid	Medium

6.3 Long-Term (9+ Months)

Feature	Description	Priority
YOLO v8 Integration	Fine-tuned object detection for specific defect types on conveyor belt	Medium
ERP Integrations	QuickBooks, SAP, Xero connectors for accounting sync	Medium
Audit Trail	Immutable change history for ISO 9001 / BSCI compliance	High
IoT + Vision	Machine sensor data combined with camera feeds for real-time QC	Low
Vision Model Training	Fine-tune moondream/llava on garment defect dataset for higher accuracy	Low

7. Appendix – Full Tech Stack

Layer	Technology	Version	Purpose
Backend	FastAPI	0.110+	REST API + WebSocket server
Backend	SQLAlchemy	2.0+	Async ORM + query builder
Backend	Alembic	1.13+	Database schema migrations
Database	PostgreSQL	16+	Primary relational data store
Database	pgvector	0.2.5+	1536-dim vector similarity search
AI Text	OpenAI GPT-4o	Latest	Intent classification + RAG answer generation
AI Vision	OpenAI GPT-4o Vision	Latest	Image defect detection, label reading, QC inspection
AI Vision	Ollama moondream	latest	Local vision fallback (~1.6 GB, offline capable)
AI Vision	Ollama llava	latest	Secondary local vision fallback (~4 GB)
AI Text	Ollama llama3.1	8b	Local text LLM fallback
AI Embed	text-emb-3-small	Latest	Document chunk embeddings (1536 dims)
AI Embed	nomic-embed-text	Local	Offline embedding fallback
Frontend	React	18	UI component framework
Frontend	TypeScript	5+	Type-safe JavaScript
Frontend	Vite	5+	Build tool + HMR dev server
Frontend	Tailwind CSS	v4	Utility-first CSS framework
Frontend	Zustand	4+	Global state management
Auth	python-jose	3.3+	JWT encoding/decoding
Auth	passlib + bcrypt	1.7+	Secure password hashing
DevOps	Docker Compose	Latest	Full-stack containerisation
DevOps	Nginx	Latest	Production reverse proxy